DOCKER

Complete Study Notes — Docker Certified Associate (DCA) Preparation

Allen | Cognizant DevOps Trainee | Azure-Focused Environment

PHASE 1 — THE MENTAL MODEL: Why Docker Exists

The Problem Docker Solves

Before Docker, installing software was painful. You would download an installer (.exe), run it, and often get errors about missing DLLs or dependencies. You would troubleshoot, install missing libraries, rerun the installer, and face a fresh error. This cycle was the reality for developers too — a developer writing software in Java might find it works on Windows but fails on Linux or Mac, because each OS has different libraries, file paths, and environments. This is called the "it works on my machine" problem.

Docker solves this by bundling the application and ALL its dependencies into a single portable unit called a container. The developer builds the container once and runs it anywhere — on Windows, Linux, Mac, in a data center, or in the cloud — with the guarantee that it behaves identically everywhere.

⚠ NOTE: Docker follows the "Build Once, Run Anywhere" approach. The Docker daemon must be installed on the target OS, but the container itself is OS-agnostic.

Core Terminology

Docker Image

A Docker image is a file that contains all the necessary dependencies, configurations, and application code needed to run a program. Think of it like a frozen, packaged template. The image itself is not running — it is just the blueprint. When you download Nginx from Docker Hub, you are downloading an image.

Docker Container

A Docker container is a running instance of an image. The relationship between image and container is the same as between a class and an object in programming, or between a recipe and a cooked meal. You can run multiple containers from the same image simultaneously.

Docker Hub

Docker Hub (hub.docker.com) is the default public registry where Docker images are stored. Anyone can push and pull images. Official images (like nginx, ubuntu, mysql) are verified by Docker. When you run docker pull nginx, Docker fetches the image from Docker Hub.

Concept	Analogy	Technical Definition
Docker Image	Recipe / Blueprint / Class	File with app + all dependencies + config
Docker Container	Cooked meal / Object / Running process	A running instance of an image
Docker Hub	App Store / GitHub	Public cloud registry for Docker images
Docker Daemon	Engine in your car	Background service that manages containers

PHASE 2 — CORE BUILDING BLOCKS

Docker Installation

Docker on Linux (Ubuntu) — Step by Step

The installation process on Ubuntu involves four main steps: adding prerequisites, adding Docker's GPG key for security, configuring Docker's APT repository, and then installing Docker CE (Community Edition).

```
# Step 1: Update and install prerequisites
apt-get update
apt-get install apt-transport-https ca-certificates curl gnupg lsb-release

# Step 2: Add Docker's official GPG key
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | gpg --dearmor ...

# Step 3: Add repository
echo "deb [arch=amd64 ...] https://download.docker.com/linux/ubuntu ..." >
/etc/apt/sources.list.d/docker.list

# Step 4: Install Docker
apt-get update
apt-get install docker-ce docker-ce-cli containerd.io

# Verify
systemctl status docker
```

```
docker info
```

Docker Desktop (Windows/Mac)

Docker Desktop is a GUI application for Windows and Mac. On Windows it supports two backends: WSL2 (Windows Subsystem for Linux 2) and Hyper-V. If WSL2 has issues, enable Hyper-V from Windows Features and switch backends in Docker Desktop settings. Docker Compose comes pre-installed with Docker Desktop — you do NOT need to install it separately on Windows or Mac.

Docker Enterprise Edition (Docker EE) on CentOS

```
# Get the repo file
wget <DOCKER_URL>/centos/docker-ee.repo -O /etc/yum.repos.d/docker-ee.repo

# Export Docker URL
export DOCKERURL="<your-docker-ee-url>"
sh -c 'echo "$DOCKERURL/centos" > /etc/yum/vars/dockerurl'

# Install
yum install docker-ee docker-ee-cli containerd.io
systemctl start docker
```

⚠ NOTE: Docker EE is the commercial/enterprise version of Docker. It ships with UCP (Universal Control Plane) and DTR (Docker Trusted Registry). Community Edition (CE) is free and open-source.

Container Identification

Every Docker container has three ways to be identified. All three can be used interchangeably in Docker commands like docker stop, docker start, docker rm, docker exec.

- **Long UUID** — Full UUID — a long hex string like 7a1b2c3d4e5f...
- **Short UUID** — First 12 characters of the long UUID (shown by docker ps)
- **Container Name** — Human-readable name specified with --name flag. If not specified Docker auto-generates one

```
# Using container name
docker stop my-nginx

# Using short UUID
docker stop 7a1b2c3d4e5f

# Using full UUID
```

```
docker stop 7a1b2c3d4e5f6a7b8c9d0e1f2a3b4c5d6e7f8a9b0c1d2e3f4a5b6c7d8e9f
```

docker run — The Master Command

docker run is the primary command to create and start a container. It combines docker pull (if image not local), docker create, and docker start into one step. Understanding all its flags is critical for both interviews and the DCA exam.

```
docker run [OPTIONS] IMAGE [COMMAND]

# Common usage example
docker run -d --name my-nginx -p 8080:80 nginx
```

Key Flags — Line by Line

`-d` → Detached mode — runs container in background, returns you to terminal immediately

`--name my-nginx` → Assigns the name "my-nginx" to the container for easy identification

`-p 8080:80` → Binds host port 8080 to container port 80 (host:container)

`-it` → Combination of -i (interactive, keeps stdin open) and -t (allocates a pseudo-TTY terminal)

`-i` → Keep stdin open even if not attached — lets you type into the container

`-t` → Allocate a pseudo-TTY — gives you a proper terminal prompt inside the container

`-v /host:/container` → Mounts a host directory as a bind mount inside the container

`--rm` → Automatically removes the container when it exits

`-e KEY=VALUE` → Sets an environment variable inside the container

`--network bridge` → Connects container to the specified Docker network

⚠ NOTE: -i and -t are TWO SEPARATE FLAGS with different functions. -i keeps stdin open (so commands reach the process), -t gives you a proper terminal display. You almost always use both together as -it when you need an interactive shell.

Attached vs Detached Mode

When you run docker run without -d, the container starts in attached (foreground) mode. Your terminal is locked to the container output — you see logs, stdout, stderr in real time. You cannot type other commands. Press Ctrl+C to stop the container.

With -d (detached mode), the container runs in the background and you immediately get your terminal back. Docker prints the full container UUID. Use docker logs <name> to see output later.

```
# Attached (foreground) mode — terminal is blocked
```

```
docker run --name attached-test -p 80:80 nginx

# Detached mode — terminal is returned immediately
docker run -d --name detached-test -p 80:80 nginx

# View logs of a detached container
docker logs detached-test

# Follow logs in real time
docker logs -f detached-test
```

Port Binding

By default, Docker containers run in an isolated network. Nobody outside can reach port 80 of your Nginx container directly because the container has a private IP address (in the 172.17.x.x range). Port binding creates a "tunnel" from a host port to a container port so external traffic can reach the container.

```
# Bind host port 8080 to container port 80
docker run -d -p 8080:80 --name my-nginx nginx

# Verify the port binding
netstat -lp     # shows docker-proxy listening on port 8080

# The format is always: -p HOST_PORT:CONTAINER_PORT
# Any request to host:8080 is forwarded to container:80
```

⚠ NOTE: Docker creates a docker-proxy process for each port binding. EXPOSE in a Dockerfile only documents which port the app listens on inside the container — it does NOT automatically publish it to the host. You must use -p to actually expose it externally.

PHASE 3 — DOCKERFILE: Building Custom Images

What is a Dockerfile?

A Dockerfile is a text file containing a series of instructions that Docker reads top to bottom to build an image. Every instruction creates a new layer in the image. The resulting image can be used to launch containers. Docker builds images with the docker build command.

```
# Build an image from Dockerfile in current directory
docker build -t my-image:1.0 .

# Tag format: repository:tag
# "." means look for Dockerfile in current directory (build context)
```

Dockerfile Instructions — Complete Reference

FROM

Every Dockerfile must begin with FROM. It sets the base image. All subsequent instructions build on top of this base. The scratch image is an empty base (no OS) used for ultra-minimal containers.

```
FROM ubuntu:22.04
FROM nginx:latest
FROM scratch        # empty base — no OS at all
```

RUN

RUN executes commands during the IMAGE BUILD process. Use it to install packages, create directories, download files, etc. Each RUN instruction creates a new image layer. Chaining commands with && reduces layers.

```
RUN apt-get update && apt-get install -y curl wget
RUN mkdir /app && chmod 755 /app
```

CMD

CMD defines the default command that runs when a container starts. It is the default starting process (PID 1). If the user provides a command in docker run, it OVERRIDES CMD. Only one CMD is allowed per Dockerfile — the last one wins.

```
# Exec form (preferred)
CMD ["nginx", "-g", "daemon off;"]

# Shell form
CMD nginx -g "daemon off;"
```

⚠ NOTE: CMD is OVERRIDABLE at runtime. If you do "docker run my-image bash", the CMD is ignored and bash is run instead.

ENTRYPOINT

ENTRYPOINT also sets the starting command, but it is NOT easily overridden. It is used when you want the container to always behave like a specific executable. CMD then acts as the default arguments to ENTRYPOINT.

```
ENTRYPOINT ["python", "app.py"]
CMD ["--port", "8080"]   # default args, can be overridden
```

	CMD	ENTRYPOINT
Purpose	Default command	Fixed executable
Overridable?	Yes, docker run <image> <cmd>	No (use --entrypoint to override)
Relationship	Can work alone	CMD provides default args to ENTRYPOINT

COPY and ADD

COPY copies files from the build context (your local machine) into the image. ADD does the same but also supports URLs and will auto-extract tar archives. Best practice is to use COPY unless you specifically need ADD's extra features.

```
COPY src/ /app/src/
COPY config.json /etc/myapp/config.json

ADD https://example.com/file.tar.gz /opt/    # ADD supports URLs + auto-extract
```

EXPOSE

EXPOSE declares which port the application inside the container listens on. This is a documentation instruction. It tells users of the image which port to publish. It does NOT actually open or map any port by itself.

```
EXPOSE 80
EXPOSE 9080

# After building, docker ps shows the exposed port under PORTS column
# docker inspect <container> shows it under ExposedPorts
```

⚠ NOTE: EXPOSE is documentation only. Without -p at runtime, the port is NOT accessible from outside the container. Think of EXPOSE as a label telling people "this app runs on port X, publish it accordingly."

HEALTHCHECK

HEALTHCHECK tells Docker how to test whether the application inside the container is working correctly. Docker monitors the container process by default (if PID 1 dies, container exits), but that is a basic check. HEALTHCHECK allows custom application-level health verification.

```
# Basic syntax
HEALTHCHECK [OPTIONS] CMD <command>

# Example: check every 30s if web server responds
HEALTHCHECK --interval=30s --timeout=10s --retries=3 CMD curl -f http://localhost/
|| exit 1

# Disable inherited health check
HEALTHCHECK NONE
```

HEALTHCHECK Options — Default Values

`--interval=30s` → How often to run the health check (default: 30 seconds)

`--timeout=30s` → How long to wait for a check to complete (default: 30 seconds)

`--start-period=0s` → Grace period before health checks start (default: 0 seconds)

`--retries=3` → How many consecutive failures before marking unhealthy (default: 3)

HEALTHCHECK Exit Codes

- **0** — Command succeeded — container is healthy
- **1** — Command failed — container is unhealthy
- **2** — Reserved — do NOT use this exit code

The health status lifecycle: starting → (after first successful check) healthy. If retries fail consecutively, status moves to unhealthy. Both docker ps and docker inspect show this status.

```
# Check health status
docker ps            # HEALTH column shows starting/healthy/unhealthy

# Detailed health logs
docker inspect <container>   # shows health check log with exit codes

# Override health check at docker run time
docker run --health-cmd="curl -f localhost" --health-interval=5s --health-
retries=3 nginx
```

⚠ NOTE: DCA exam favourite: If interval=30s and retries=3, the health check takes at least 90 seconds (3 x 30s) before declaring a container unhealthy. If interval=10s, timeout=5s, retries=3 — this can determine health within 30s.

ENV, WORKDIR, USER, ARG, LABEL

```
ENV APP_PORT=8080
WORKDIR /app
USER appuser
ARG VERSION=1.0
LABEL version="1.0"
```

Multi-Stage Builds

Multi-stage builds allow you to use multiple FROM instructions in a single Dockerfile. This is a powerful pattern for reducing final image size. The first stage (builder) can have all compilation tools. The final stage (runtime) only copies the compiled artifacts — no build tools, no source code.

```
# Stage 0: build stage (golang environment)
FROM golang:1.21 AS builder
WORKDIR /app
COPY . .
RUN go build -o myapp .

# Stage 1: runtime stage (tiny empty image)
FROM scratch
COPY --from=0 /app/myapp /myapp   # --from=0 references stage 0
CMD ["/myapp"]
```

`FROM golang:1.21 AS builder` → First FROM = first stage. Named "builder" for easy reference

`FROM scratch` → Second FROM = second (final) stage. Scratch is an empty image

`COPY --from=0` → Copy artifact from stage 0 (index) into this stage. Can also use --from=builder (name)

⚠ NOTE: Multi-stage builds' primary purpose is to allow artifacts from one image to be used in another, resulting in a much smaller final image. Only the final stage becomes the actual image — all previous stages are discarded.

PHASE 4 — DOCKER NETWORKING

Docker Network Drivers

Docker networking is handled through pluggable drivers. Each driver provides different network behaviour. The driver you choose depends on whether you need container isolation, host-level performance, multi-host communication, or Swarm-wide networking.

Driver	Use Case	Key Behaviour
bridge	Default for standalone containers	Containers get 172.17.x.x IPs; communicate via docker0 bridge
host	Max performance, no isolation	Container shares host network directly; no port mapping needed
none	Complete isolation	No network at all; container cannot communicate
overlay	Multi-host Swarm networking	Spans multiple Docker hosts; required for Swarm services
macvlan	Legacy app needs MAC address	Container appears as physical device on the network

Bridge Network — Default Behaviour

When you run a container without specifying a network, it is attached to the default bridge network (docker0). The container receives a private IP address in the 172.17.x.x range. Containers on the same bridge network can communicate with each other using their IP addresses. The docker0 interface on the host acts as a virtual switch connecting all containers.

```
# List all networks
docker network ls

# Create a custom bridge network
docker network create my-network

# Run container on specific network
docker run -d --network my-network --name app1 nginx
```

```
# Connect existing container to a network
docker network connect my-network existing-container

# Inspect a network
docker network inspect my-network
```

⚠ NOTE: Containers on a custom bridge network can resolve each other by NAME (DNS). Containers on the DEFAULT bridge network (docker0) can only communicate by IP address, not by name.

Port Binding Deep Dive

Port binding works via iptables rules and the docker-proxy process. When you use -p 8080:80, Docker creates an iptables PREROUTING rule and a docker-proxy listener. Any TCP/UDP traffic hitting the host on port 8080 is forwarded to the container's port 80.

```
# Basic port binding
docker run -d -p 8080:80 nginx

# Bind to specific host IP
docker run -d -p 127.0.0.1:8080:80 nginx

# Multiple ports
docker run -d -p 80:80 -p 443:443 nginx

# Verify
netstat -lp    # docker-proxy on 8080
docker ps      # shows 0.0.0.0:8080->80/tcp
```

PHASE 5 — VOLUMES AND STORAGE

Why Volumes?

Containers are ephemeral by design. When a container is removed, all data inside it is gone. For databases, file uploads, logs, and any data that needs to persist across container restarts or removals, you need volumes or bind mounts.

Type	Location	Managed By	Use Case
Volume	/var/lib/docker/volumes/	Docker	Preferred — persistent app data, databases
Bind Mount	Any host path	User/OS	Development — share source code into container
tmpfs	Host RAM (Linux only)	Kernel	Sensitive data — secrets, temporary, high speed

Docker Volume Commands

```
# Create a volume
```

```
docker volume create my-data

# List volumes
docker volume ls

# Inspect a volume (shows mount path)
docker volume inspect my-data

# Remove a volume
docker volume rm my-data

# Use volume when running container
docker run -d -v my-data:/var/lib/mysql mysql:8.0

# Bind mount (host path)
docker run -d -v /home/allen/data:/app/data nginx

# Using --mount flag (more explicit)
docker run -d --mount type=volume,source=my-data,target=/data nginx
```

⚠ NOTE: Named volumes persist even after docker container rm. To clean up volumes, use docker volume rm <name> or docker volume prune. Data in volumes is NOT deleted when the container is removed.

PHASE 6 — DOCKER SWARM: Orchestration

Swarm Concepts

Docker Swarm is Docker's built-in container orchestration system. It lets you manage a cluster of Docker hosts (nodes) as a single virtual system. Swarm handles automatic container placement, load balancing, scaling, rolling updates, and self-healing.

- **Manager Nodes** — Hosts that control the swarm cluster. Maintain cluster state via Raft consensus. Must have an odd number for quorum (1, 3, 5...)
- **Worker Nodes** — Hosts that run the actual container workloads (tasks)
- **Service** — A definition of how containers should run — image, replicas, ports, constraints
- **Task** — A single running container within a service (one slot)

⚠ NOTE: Raft consensus requires a quorum: majority of managers must be healthy for the cluster to function. With 3 managers you can lose 1. With 5, you can lose 2.

Swarm Initialization and Management

```
# Initialize swarm on first manager node
```

```
docker swarm init --advertise-addr <MANAGER_IP>
# Output gives join tokens

# Get join token for workers
docker swarm join-token worker

# Get join token for managers
docker swarm join-token manager

# Join as worker (run on worker node)
docker swarm join --token SWMTKN-xxx <MANAGER_IP>:2377

# List nodes in the swarm
docker node ls

# Promote worker to manager
docker node promote <node-id>

# Demote manager to worker
docker node demote <node-id>

# Leave the swarm
docker swarm leave --force   # on manager
```

Docker Services — Creating and Managing

```
# Create a service with 2 replicas
docker service create --name my-web --replicas 2 --publish 80:80 nginx

# List services
docker service ls

# See tasks (containers) of a service
docker service ps my-web

# Scale a service
docker service scale my-web=5

# Update a service (rolling update)
docker service update --image nginx:latest my-web

# Inspect a service (pretty output)
docker service inspect --pretty my-web

# Remove a service
docker service rm my-web
```

Publishing Ports in Swarm

When a service is created with a published port in Swarm, the ingress routing mesh ensures that port is accessible on EVERY node in the cluster, regardless of which node the task is actually running on. This is different from standalone container port binding.

```
# Publish port 80 for a swarm service
docker service create --name web --replicas 2 --publish 80:80 nginx

# Now any node IP on port 80 reaches the service
curl http://<any-node-ip>:80
```

Swarm Locking (autolock)

Swarm stores sensitive data: the TLS keys used to encrypt node communications, and the keys to encrypt/decrypt Raft logs on disk. By default these keys are stored in plaintext. If an attacker gets access to the manager node filesystem, they can read these keys. Swarm autolock encrypts these keys.

```
# Enable autolock on existing swarm
docker swarm update --autolock=true
# Save the unlock key that is displayed!

# After restart, Docker is locked
docker node ls     # ERROR: swarm is encrypted and needs to be unlocked

# Unlock with the saved key
docker swarm unlock
# Enter the unlock key at the prompt

# Rotate the unlock key
docker swarm ca --rotate
```

⚠ NOTE: Save the autolock unlock key somewhere safe when you enable it. If you lose it and the Docker daemon restarts, you cannot unlock the swarm and will lose access to the cluster.

Inspecting Services and Nodes

```
# Inspect a service (raw JSON)
docker service inspect my-web

# Pretty output — shows constraints, replicas, image etc.
docker service inspect --pretty my-web

# Inspect a specific node
docker node inspect <node-id> --pretty
# Shows: hostname, status, availability, CPU, RAM, Docker version, IP

# Troubleshoot why tasks are in pending state
docker service inspect --pretty my-web
# Look for: Placement Constraints section
```

The --pretty flag makes service inspect much more readable. It shows placement constraints, which is a common reason tasks go to pending state. For example, a typo in a node label

constraint like "region==Mumbai" instead of "region==mumbai" will cause all tasks to remain pending.

PHASE 7 — DOCKER COMPOSE AND DOCKER STACK

Docker Compose

Docker Compose is a tool for defining and running multi-container applications. Instead of running multiple docker run commands manually, you define all services, networks, and volumes in a single docker-compose.yml file. With one command you bring everything up, with another you tear it all down.

```
# Install on Linux (not needed on Windows/Mac)
curl -L "https://github.com/docker/compose/releases/download/v2.x.x/docker-compose-$(uname -s)-$(uname -m)" -o /usr/local/bin/docker-compose
chmod +x /usr/local/bin/docker-compose

# Start all services (detached)
docker compose up -d

# Stop and remove all containers
docker compose down

# Validate the YAML syntax
docker compose config

# View running compose services
docker compose ps
```

docker-compose.yml — Example (Nginx + Redis)

```
version: "3"

services:
  webserver:
    image: nginx
    ports:
      - "8080:80"

  database:
    image: redis
```

docker-compose.yml — Example (WordPress + MySQL)

```
version: "3"
```

```yaml
services:
  db:
    image: mysql:5.7
    restart: always
    environment:
      MYSQL_ROOT_PASSWORD: example

  wordpress:
    image: wordpress:latest
    depends_on:
      - db
    ports:
      - "8000:80"
    restart: always
    environment:
      WORDPRESS_DB_HOST: db
      WORDPRESS_DB_PASSWORD: example
```

`version: "3"` → Compose file format version — determines which features are available

`services:` → Top-level key where all containers/services are defined

`image: nginx` → Use this Docker image to create the service container

`ports: - "8080:80"` → Bind host port 8080 to container port 80

`depends_on: - db` → Start db service before this service

`restart: always` → Restart policy — restart this container if it stops for any reason

`environment:` → Inject environment variables into the container

Docker Stack — Deploying Compose to Swarm

Docker Compose deploys containers only on the local Docker host. If you want to deploy your multi-service application across a Swarm cluster (across multiple nodes), you use docker stack deploy. Stack takes your docker-compose.yml and deploys it as a collection of Swarm services.

```bash
# Deploy a stack to the swarm
docker stack deploy --compose-file docker-compose.yml my-stack

# List all stacks
docker stack ls

# List tasks in a stack
docker stack ps my-stack

# Remove a stack
docker stack rm my-stack
```

	docker compose up	docker stack deploy

Scope	Single host only	Entire Swarm cluster
Command	docker compose up -d	docker stack deploy --compose-file ...
Use case	Local development, testing	Production multi-node deployment
Requires Swarm?	No	Yes (docker swarm init first)

PHASE 8 — DOCKER SECURITY

Docker Group — Non-Root Users

The Docker daemon runs as root and communicates via a Unix socket (/var/run/docker.sock). By default only root can communicate with this socket. If a non-root user tries docker ps, they get "permission denied". The recommended way to allow non-root users is to add them to the docker group.

```
# Add user to docker group
usermod -aG docker <username>

# Verify group membership
id <username>

# The user must log out and back in for group changes to take effect

# Verify docker group exists
cat /etc/group | grep docker
```

⚠ NOTE: Do NOT add users to the sudoers file just to run Docker. Add them to the docker group instead. Being in the docker group is sufficient and does not give full sudo/root privileges to the system.

Linux Capabilities

Linux capabilities break down the monolithic root privilege into fine-grained units. For example, the ability to set immutable bits on files is one capability (LINUX_IMMUTABLE). Docker containers are assigned a default set of capabilities at startup. Capabilities NOT in the default set are unavailable to the container, even if you are "root" inside the container.

```
# Add a capability when running a container
docker run --cap-add LINUX_IMMUTABLE amazon-linux bash

# Drop a capability from a container
docker run --cap-drop NET_RAW nginx

# Multiple capabilities
docker run --cap-add SYS_PTRACE --cap-drop NET_RAW nginx
```

Example: inside a default container, running chattr +i test.txt (sets immutable bit) fails with "Operation not permitted" even though you are root, because LINUX_IMMUTABLE is not in the default capability set. Adding --cap-add LINUX_IMMUTABLE makes it work.

⚠ NOTE: Capabilities = Principle of Least Privilege. Only give containers the capabilities they actually need. This reduces the blast radius if a container is compromised.

Privileged Containers

A privileged container is given ALL Linux capabilities AND direct access to all host devices (/dev/*). It bypasses the capability restrictions entirely. A privileged container has nearly the same access to the host as a process running directly on the host OS.

```
# Run a privileged container
docker run --privileged -it amazon-linux bash

# Inside privileged container: ALL host devices are visible
ls /dev    # shows vda, vdb, sda etc. just like on the host

# Capabilities are unrestricted in privileged containers
chattr +i test.txt    # works — LINUX_IMMUTABLE is available
```

	Normal Container	Privileged Container
Capabilities	Default subset only	ALL capabilities
Host devices (/dev/*)	Not accessible	Full access
Security	Isolated	Near-host level access
Use case	All standard workloads	Docker-in-Docker, kernel operations
Risk	Low	High — should be restricted

⚠ NOTE: Never run privileged containers in production without strong justification. If a privileged container is compromised, the attacker effectively has root on the host. Use --cap-add for individual capabilities instead.

Container Security Scanning (DTR)

Docker Trusted Registry (DTR) has a built-in security scanning feature that analyzes container images for known vulnerabilities. It checks the image layers against CVE (Common Vulnerabilities and Exposures) databases and reports findings.

- Scans image layers against CVE databases automatically when images are pushed
- Shows clean (no vulnerabilities) or a list of CVEs with severity
- Can be configured to trigger on push or run manually

DTR Webhooks

DTR can send HTTP POST notifications (webhooks) to external systems when certain events occur. This enables CI/CD integration — DTR tells Jenkins when a security scan is complete, and Jenkins can then pull the image for further processing like unit testing.

- **Location** — Configured per repository (not globally)
- **Event types** — "Tag pushed to repository" or "Security scan completed"
- **Webhook URL** — URL of Jenkins, build system, or any HTTP endpoint

⚠ NOTE: For the DCA exam: webhooks are under the repository settings. Know the event type "security scan completed" for the CI/CD integration scenario.

PHASE 9 — DOCKER ENTERPRISE: UCP

Universal Control Plane (UCP)

Universal Control Plane is Docker's enterprise-grade cluster management GUI. It runs as a containerized application on top of Docker EE. Instead of managing your swarm cluster via CLI commands, UCP gives you a web interface to manage containers, services, networks, volumes, users, and permissions.

UCP Minimum Requirements

- **Docker EE** — Docker Enterprise Edition must be running on all nodes
- **Manager nodes** — 8 GB RAM (recommended: 16 GB)
- **Worker nodes** — 4 GB RAM
- **Disk — managers** — 5 GB free disk space for /var partition (manager)
- **Disk — workers** — 500 MB free disk space (workers)

UCP Client Bundles

A UCP Client Bundle is a set of TLS certificates that you download from UCP and configure on your local machine. Once configured, any Docker CLI command you run locally is actually executed on the remote UCP cluster. This is similar to kubeconfig in Kubernetes or AWS access keys for the AWS CLI.

```
# Steps to use a client bundle:
# 1. Login to UCP web console
# 2. Go to user profile → Client Bundle
# 3. Click "New Client Bundle" → download ZIP
# 4. Extract the ZIP on your local machine
```

```
# 5. Source the environment script
source env.sh

# Now your local docker commands run against the remote UCP
docker service create --name remote-nginx --replicas 2 nginx
# This service is created on the UCP cluster, not your laptop
```

⚠ NOTE: Client bundles are USER-SPECIFIC. The permissions of the client bundle match the permissions of the UCP user it was generated for. An admin bundle has full cluster access; a dev bundle might only access specific namespaces.

PHASE 10 — KUBERNETES (DCA Exam Context)

kubectl Basics

kubectl is the command-line tool for interacting with a Kubernetes cluster. The pattern is very similar to Docker CLI — docker run becomes kubectl run, docker ps becomes kubectl get pods. The key concept is that in Kubernetes, the unit of deployment is a Pod (not a bare container).

Docker Command	kubectl Equivalent	Purpose
docker run -d --name app nginx	kubectl run app --image=nginx	Create and run a container/pod
docker ps	kubectl get pods	List running containers/pods
docker exec -it app bash	kubectl exec -it app -- bash	Shell into container/pod
docker stop/rm app	kubectl delete pod app	Remove container/pod
docker inspect app	kubectl describe pod app	Detailed info about container/pod

Kubernetes Services

A Kubernetes Service solves the problem of pod IP addresses changing dynamically. When a pod crashes and a new one starts, the new pod gets a different IP. Any frontend app hardcoding the old IP breaks. A Service provides a stable IP address and DNS name that front-ends communicate with, and routes traffic to the healthy backend pods.

```
# Create service from YAML
kubectl apply -f service.yaml

# List services
kubectl get service

# Describe service (see endpoints)
kubectl describe service my-service

# Get all pods from all namespaces
kubectl get pods --all-namespaces

# Get pods with a specific label
kubectl get pods -l environment=development
```

```
# or
kubectl get pods --selector environment=development
```

Service Types

- **ClusterIP** — Internal cluster IP only. Not reachable from outside the cluster. DEFAULT type.
- **NodePort** — Exposes service on each node's IP at a static port. Accessible externally via NodeIP:NodePort.
- **LoadBalancer** — Creates a cloud load balancer (works on AKS, GKE, EKS etc.)
- **ExternalName** — Maps a service to an external DNS name

⚠ NOTE: ClusterIP is the DEFAULT service type. If you create a service without specifying a type, it gets ClusterIP. ClusterIP services are only reachable from within the cluster.

Labels, Selectors, and Service Endpoints

Labels are key-value pairs attached to Kubernetes objects (pods, deployments, nodes). Selectors allow services to automatically discover and connect to all pods with matching labels. This is how services scale dynamically — new pods with the correct label are automatically added to the service's endpoint list.

```
# Service YAML with selector
apiVersion: v1
kind: Service
metadata:
  name: backend-gateway
spec:
  selector:
    app: nginx       # Select all pods with label app=nginx
  ports:
    - port: 80       # Service listens on port 80
      targetPort: 80 # Routes to pod port 80
```

`port: 80` → The port THIS SERVICE listens on

`targetPort: 80` → The port on the PODS where traffic is forwarded

`selector: app: nginx` → Automatically include all pods with this label as endpoints

PHASE 11 — DCA EXAM IMPORTANT POINTERS

HEALTHCHECK Default Values (Must Know)

Option	Default Value	What It Controls
--interval	30 seconds	How often health check runs
--timeout	30 seconds	Max time for one health check to complete
--start-period	0 seconds	Grace period before checks start
--retries	3	Consecutive failures before marking unhealthy

Exam trap: interval=30s, retries=3 means it takes at LEAST 90 seconds before a container is marked unhealthy. If the exam asks "which configuration determines health within 1 minute?", look for interval*retries < 60 seconds.

Storage Classes and PVC/PV (Kubernetes)

- **Storage Class** — Storage Class describes the type and class of storage offered (standard = slow, premium = fast)
- **Volume Expansion** — allowVolumeExpansion: true must be set in StorageClass to allow PVC expansion
- **Expansion Steps** — Steps: 1) Enable allowVolumeExpansion in StorageClass 2) Resize PVC 3) Restart pod

Reclaim Policies

- **Retain** — PVC deleted → PV moves to "Released" state. Data still exists. Admin must manually recover it before PV can be reused.
- **Delete** — Dynamically provisioned PV is automatically deleted when PVC is deleted
- **Recycle** — PV is scrubbed (rm -rf) and made available for new claims. Deprecated feature.

⚠ NOTE: RETAIN: Even if PVC is deleted, the PV still exists but is in "Released" state. It is NOT immediately available for another claim because old data remains.

Volume vs Persistent Volume (Kubernetes)

	Volume	Persistent Volume (PV)
Lifecycle	Tied to the Pod lifecycle	Independent of Pod lifecycle
When pod is deleted	Volume is also deleted	PV persists (based on reclaim policy)
Use case	Temporary/shared storage within a pod	Databases, stateful applications

kubectl Selector Flags

```
# List pods from ALL namespaces
kubectl get pods --all-namespaces

# Filter by label — short form
kubectl get pods -l environment=development

# Filter by label — long form (both are correct answers in exam)
kubectl get pods --selector environment=development

# List pods with labels shown
kubectl get pods --show-labels
```

⚠ NOTE: -l and --selector are IDENTICAL. Both are valid answers in DCA exam questions about label-based filtering.

Multi-Stage Build Key Points

- **Purpose** — Primary purpose is to copy compiled artifacts from one stage to another, keeping the final image small
- **Cross-stage copy** — COPY --from=0 references stage 0 (first FROM). Can also use --from=builder if named.
- **Final stage only** — Only the LAST stage becomes the actual image. All previous stages are discarded after build.
- **Typical pattern** — Golang example: compile in golang stage → copy binary to scratch stage → ultra-small final image

DCA Exam Domain Summary

Domain	Key Topics
Domain 1: Orchestration	Docker Swarm, Services, Stacks, Kubernetes basics
Domain 2: Image Creation	Dockerfile instructions, docker build, tagging, registries, DTR
Domain 3: Installation	Docker CE/EE install, UCP, Docker Desktop, Docker EE requirements
Domain 4: Networking	Network drivers (bridge, overlay, host), port binding, DNS
Domain 5: Security	Capabilities, privileged containers, content trust, DTR scanning, secrets
Domain 6: Storage	Volumes, bind mounts, PV/PVC, StorageClass, reclaim policies

QUICK REFERENCE — ALL KEY COMMANDS

Container Lifecycle

```
docker run -d --name <name> -p <host>:<container> <image>
docker ps                    # running containers
docker ps -a                 # all containers (including stopped)
docker stop <name/id>
docker start <name/id>
docker restart <name/id>
docker rm <name/id>
docker rm -f <name/id>       # force remove running container
docker exec -it <name> bash
docker logs <name>
docker logs -f <name>        # follow logs
docker inspect <name>
docker stats                 # live resource usage
docker top <name>            # processes in container
```

Image Management

```
docker images
docker pull <image>:<tag>
docker push <registry>/<image>:<tag>
docker build -t <name>:<tag> .
docker tag <source> <target>
docker rmi <image>
docker history <image>       # show layer history
docker save -o file.tar <image>   # export image to tar
docker load -i file.tar           # import image from tar
```

Docker Swarm

```
docker swarm init --advertise-addr <IP>
docker swarm join-token worker
docker swarm join-token manager
docker swarm leave --force
docker swarm update --autolock=true
docker swarm unlock
docker node ls
docker node promote <id>
docker node demote <id>
docker service create --name <name> --replicas <n> --publish <h>:<c> <image>
docker service ls
docker service ps <name>
docker service scale <name>=<n>
docker service update --image <new-image> <name>
docker service inspect --pretty <name>
docker service rm <name>
docker stack deploy --compose-file <file> <stack-name>
docker stack ls
docker stack ps <stack-name>
docker stack rm <stack-name>
```

Networks and Volumes

```
docker network ls
docker network create <name>
docker network inspect <name>
docker network connect <network> <container>
docker network rm <name>
```

```
docker volume ls
docker volume create <name>
docker volume inspect <name>
docker volume rm <name>
docker volume prune           # remove all unused volumes
```

Docker Compose

```
docker compose up -d
docker compose down
docker compose config         # validate YAML
docker compose ps
docker compose logs -f
docker compose exec <service> bash
docker compose pull           # pull latest images
docker compose build          # build images defined in compose file
```